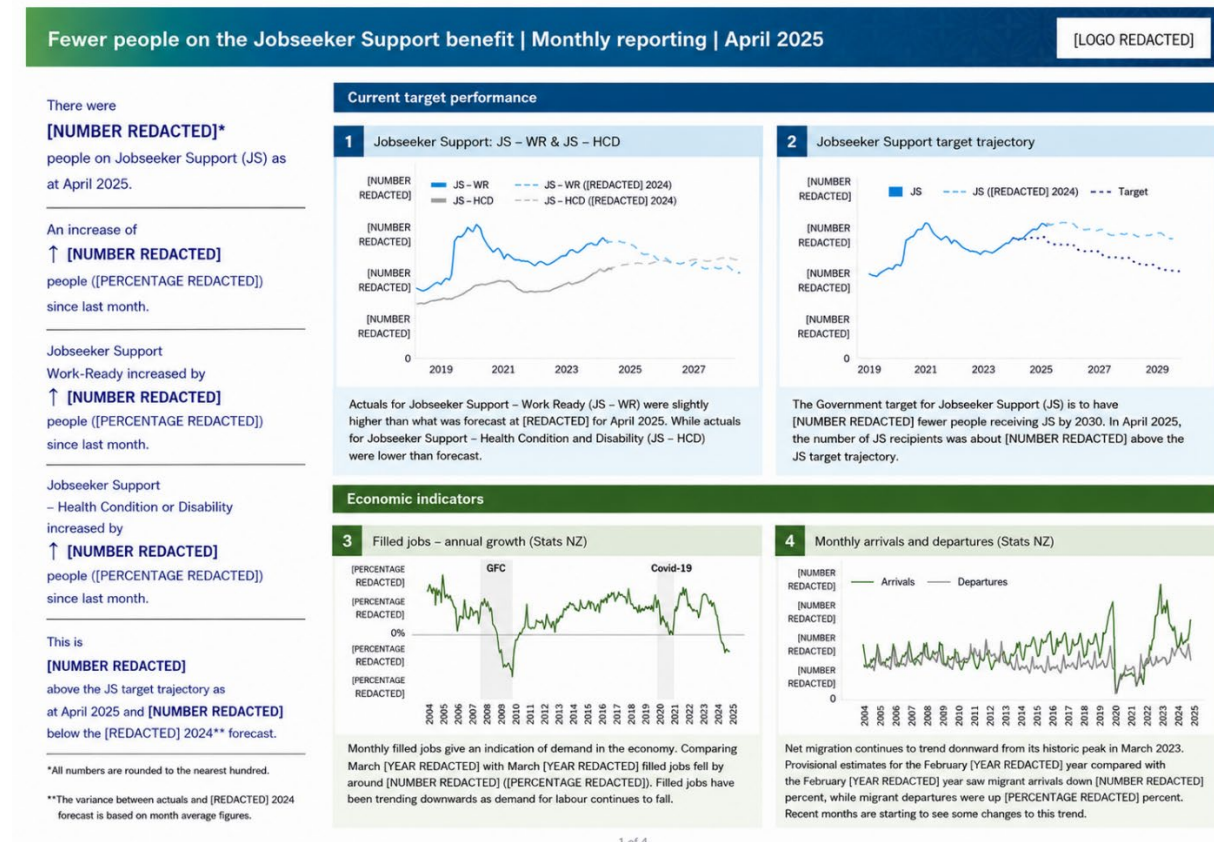


Target Reporting Automation



* Material in the snippet above has been redacted due to confidentiality reasons.

Catalyst for project

A dashboard aimed at tracking trends against a target had been commissioned and had to be put together manually every month by a collection of analysts. Once the final look and contents of the product had been decided, it was commissioned for **automation to reduce the total time and effort spent on production**.

The automation stage for this product involved extracting data from organisational databases to creating a document that could update graphs and figures easily while at the same time allowing room for commentary to be adjusted manually by an analyst.

Solution approach and design decisions

The automation solution was designed around technologies that were already well established within the organisation, enabling the reporting process to be integrated into existing operational workflows. Selecting familiar tools reduced single-person dependency by ensuring the solution could be understood, maintained and enhanced by multiple analysts across the team.

The overall architecture split the reporting process into separate stages, with each application performing a particular task. Organisational data was extracted and summarised using SAS, producing datasets that would be used downstream. R was then used to transform the summarised data into visualisations and dynamically generated text variables, providing a flexible and repeatable method for producing analytical outputs. Adobe

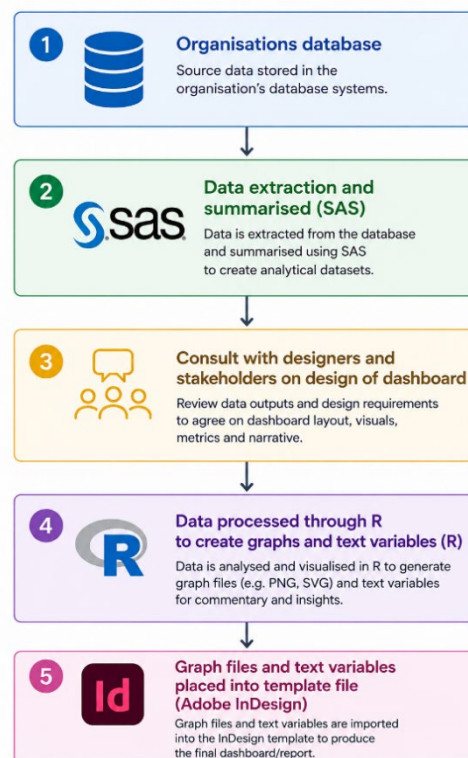
InDesign acted as the presentation layer, combining graphs, text variables and report layouts into a consistent executive reporting product.

Separating data extraction, transformation, visualisation and document production greatly improves maintainability of the process and enables changes to individual components without requiring the entire reporting pipeline to be redesigned.

A key design principle was to maximise automation while preserving analyst control where analysis or commentary was needed, which balances automation with quality assurance. Creating a reporting process that was efficient, reliable and adaptable to changes.

The final product was a 4-page 4 report with 21 graphs and 28 text elements fully automated.

Automation Design process (+ tools used at each step)



Code snippets (SAS and R)

SAS Code snippet (dataset and variables adjusted due to confidentiality)

```
proc sql;
create table JS_counts as
select date format=date9., year, month

      ,count(*) as counts
      ,sum(case when group_1 in ('relevant_code_1', 'relevant_code_2') then 1 else 0 end) as group_1_counts
      ,sum(case when group_1 in ('relevant_code_1', 'relevant_code_2') then 1 else 0 end) as group_2_counts
      ,sum(case when 18 le age le 24 then 1 else 0 end) as yr18_24
      ,sum(case when 25 le age le 34 then 1 else 0 end) as yr25_34
      ,sum(l1_maori) as maori
      ,sum(l1_european) as european
```

```

, sum(l1_pasifika) as pasifika

, sum(case when group_2 le 365 then 1 else 0 end) as dur_1t_1yr

, sum(case when 366 le group_2 le 730 then 1 else 0 end) as dur_1_2yr

, sum(case when region in ('Auckland Central') then 1 else 0 end) as akl_ctr

, sum(case when region in ('Auckland South') then 1 else 0 end) as akl_sth

, sum(case when region in ('Auckland North') then 1 else 0 end) as akl_nth

from organisational_dataset

group by date, year, month;

quit;

```

R code for creating graph 1 (uses ggplot2 package)

```

graph.ylim = c(0, 300000)
graph.breaks = as.numeric(seq(0, 300000, 50000))
graph.breaks.label = paste0(formatC(graph.breaks, format="f", big.mark=",", digits=0))

graph.x.breaks = c(0, 24, 48, 72, 96, 120) # Every 2 years (24 months apart)
graph.x.breaks.label = c(2019, 2021, 2023, 2025, 2027, 2029) # Labels for every second year

outgraph <- ggplot(mthly_js_data, aes(x=date_ind)) +
  geom_line(size=0.8, color="#3395d0", aes(y=JS_counts)) +
  geom_line(size=0.8, color= "#81bde1", aes(y=js_fc), linetype="dashed") +
  geom_line(size=0.8, color= "#1d1f6b", aes(y=js_targets), linetype="dotted")+
  theme_classic() +
  theme(panel.grid.major = element_blank(),
        panel.grid.minor = element_blank(),
        panel.background = element_rect(fill = "white", colour = NA),
        plot.background = element_rect(fill = "transparent", colour = NA) ) +
  theme(plot.margin = unit(c(0.176389, 0, 0, 0), "cm")) +
  theme(legend.position="none") +
  theme(axis.title.y = element_blank()) +
  theme(axis.title.x = element_blank()) +
  theme(axis.text=element_text(family = "verdana", size=9, colour = "#595959"))+
  theme(axis.line = element_blank()) +
  theme(axis.ticks = element_blank()) +
  theme(axis.line.x = element_blank()) +
  scale_x_continuous(labels=graph.x.breaks.label, breaks=graph.x.breaks) +
  scale_y_continuous(position="left", label=graph.breaks.label, limits=graph.ylim, expand = c(0,0), breaks=graph.breaks)

### export graph to PDF
fileout = paste(output.path, "/", "js_targ_graph_2", ".pdf", sep="")
pdf(file = fileout, width=5.43 ,height=2.76) ## pdf units are in inches
print(outgraph)
dev.off()

```